Lab 1 Booklet

GPIO and the Memory Map

Contents

1 Part "A": Inline Assembly **2**

1.1 General-purpose I/O . . . 2
1.2 Memory-mapped I/O . . . 2
1.3 Core registers and memory . . . 3
1.4 The registers of an output pin . . . 4
1.5 The mode of a pin . . . 5
1.6 Read-modify-write . . . 6
1.7 Read-modify-write and interrupts . . . 7
1.8 Inline assembly . . . 7
1.9 Time in a software delay . . . 9
1.10 Periodic signals and the oscilloscope . . . 9
1.11 Flash, SRAM, and the start of a program . . . 10
1.12 Buses and clocks . . . 11

2 Part "B": Register Access in the C Language **12**

2.1 Register access in the C language . . . 12
2.2 Push-pull outputs . . . 13
2.3 The Schmitt trigger input . . . 14
2.4 The input from button B1 . . . 14
2.5 Contact bounce . . . 16
2.6 Methods to remove contact bounce . . . 17

3 Part "C": The HAL **19**

3.1 The HAL . . . 19
3.2 Time measurement with the HAL . . . 20
3.3 A loop that does not wait . . . 21

A Morse code **22**

1 Part "A": Inline Assembly

To control an LED, a program writes to a memory address. The user LED LD2 is connected to pin PA5. Bit 5 of `GPIOA_ODR`, the output data register of port *A*, sets the output level of PA5. The address of this register is `0x4800 0014` (Figure 1). The prefix `0x` marks a hexadecimal number. Some assemblers write the same number with the suffix `h`: `48000014h`. When the clock of port *A* is on and PA5 is an output, a `1` in bit 5 turns LD2 on. The value `0` turns LD2 off. Each of these operations is a write to a register of the chip.

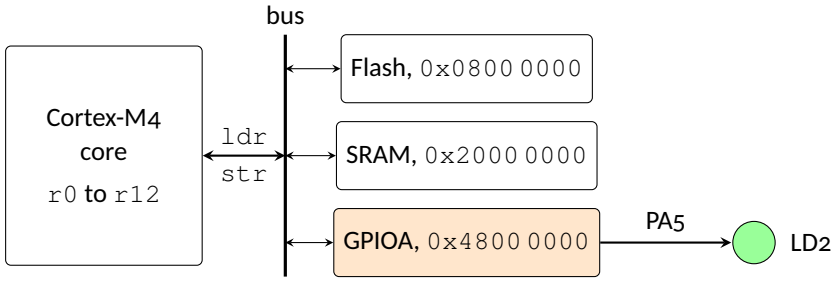


Figure 1: The Cortex-M4 core uses the same bus and the same `ldr` and `str` instructions to access flash, SRAM, and the registers of GPIO port *A*.

1.1 General-purpose I/O

The configuration of a microcontroller pin sets its function. The pin can be a *digital* output: the program sets its level to one of two values, high or low. PA5, which drives LD2, is an example. The pin can also be a digital input, or a connection to an on-chip peripheral. Thus, the pins are "general-purpose". The pins are in groups, the *ports*. Each port has a letter, from *A* to *G*, and up to sixteen pins: PA5 is pin 5 of port *A* (Figure 2). Each port has its own base address and its own set of registers.

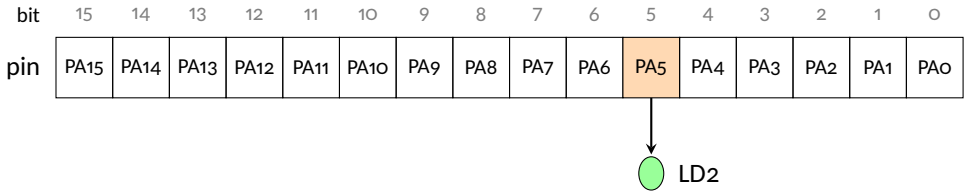


Figure 2: The sixteen pins of port *A*. Bit n of `GPIOA_ODR` sets the level of pin PAn: bit 5 sets PA5, which drives LD2.

Other microcontrollers also have GPIO ports, and their registers have the same functions under other names. One register sets the direction or the mode of each pin. One register holds the output levels, and one register gives the input levels. On the AVR microcontroller of the Arduino Uno, these registers are `DDR`, `PORT`, and `PIN`. On the STM32, they are `MODER`, `ODR`, and `IDR`.

1.2 Memory-mapped I/O

The chip has hardware blocks outside the processor core, the *peripherals*: a GPIO port is an example. The program controls a peripheral through its *peripheral registers*. Each of these registers has a memory address, in the range `0x4000 0000` to `0x5FFF FFFF`. For example, a write to a GPIO register changes the

state of a pin. The level of the pin comes from a read of the register. Because the registers have addresses, the processor accesses them as memory. The *load* instruction reads a register, and the *store* instruction changes it.

Arm, the company that designs the Cortex-M processors, divides the address space of each Cortex-M4 into the same regions (Figure 3). Each chip maker puts its flash, its SRAM, and its peripherals into these regions. Thus, the memory maps of all Cortex-M4 chips have the same shape, and only the addresses inside each region change from chip to chip. The System region holds the peripherals of the processor itself, for example its interrupt controller. These peripherals are at the same addresses on each Cortex-M4 chip.

Memory-mapped I/O is not the only method to access a peripheral. Some processors also have a separate address space for devices, with special instructions. The x86 processors of personal computers have the instructions `in` and `out` for this space.

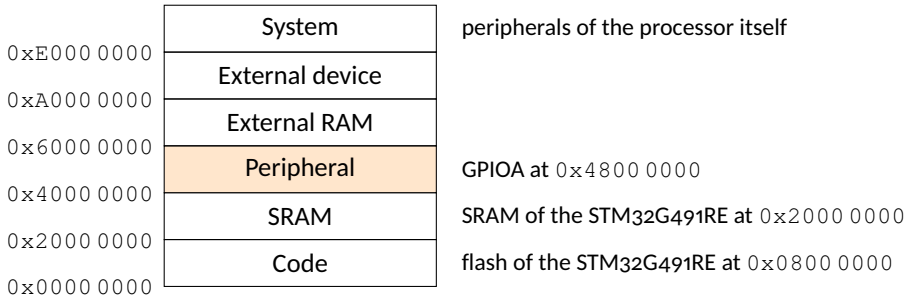


Figure 3: The regions of the address space of each Cortex-M4 processor (not to scale). Higher addresses are higher in the figure. On the right: where the STM32G491RE puts its parts.

1.3 Core registers and memory

The processor has two types of storage (Figure 1):

- *core registers*, `r0` to `r12`: thirteen 32-bit registers in the processor, with no address.
- *memory*, which the processor accesses over the bus at an address: flash from `0x0800 0000`, SRAM from `0x2000 0000`, and the memory-mapped peripherals from `0x4000 0000`.

The global variables of a C program are in SRAM: the linker gives each variable an address there.

The Cortex-M4 is a *load/store* machine. Arithmetic and logic instructions, for example `subs`, `orr`, `bic`, and `eor`, operate only on core registers. Only `ldr` and `str` access memory.

Thus, a change to a value in memory usually has three steps. The processor loads the value into a core register with `ldr`, changes it there, and stores it with `str`. This is true for a counter in SRAM and for a GPIO register.

The core has three more registers, with special functions (Figure 4):

- `r13`, the stack pointer (SP): the address of the top of the stack.
- `r14`, the link register (LR): the return address of a function call.
- `r15`, the program counter (PC): the address of the current instruction.

The program status register holds the *condition flags*. These flags record properties of the result of an instruction, for example a negative result or a carry.

Arm and RISC-V processors are load/store machines. Some other processors, for example the x86 processors of personal computers, can add a number to a value in memory with one instruction. Inside the processor, that instruction also reads the memory, changes the value, and writes it back.

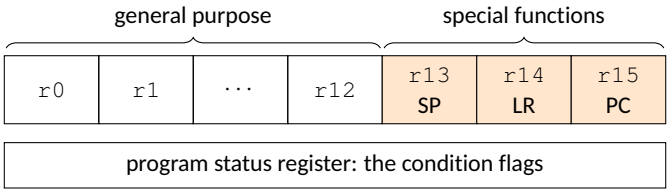


Figure 4: The registers of the Cortex-M4 core (simplified). Each register holds 32 bits and has no address.

1.4 The registers of an output pin

The address of a register of port *A* is the base address of port *A*, `0x4800 0000`, plus an offset. Three registers control an output pin of port *A* (Table 1 and Figure 5):

- **`RCC_AHB2ENR`, the clock gate.** This register is in the reset and clock control block (RCC). It enables the peripheral clocks on AHB2, one of the three Advanced High-performance Buses (AHB) of the chip. The reset value is `0x0000 0000`, thus all ports are disabled after a reset. Bit 0, GPIOAEN, enables port *A*.
- **`GPIOA_MODER`, the mode register.** It has two bits for each pin, which set the mode of the pin.
- **`GPIOA_ODR`, the output data register.** It has one bit for each pin. The bit holds the level that the pin drives as an output.

Register	Address	Purpose
`RCC_AHB2ENR`	`0x4002 104C`	Clock gate. Bit 0 (GPIOAEN) enables port *A*.
`GPIOA_MODER`	`0x4800 0000`	Pin mode, 2 bits for each pin.
`GPIOA_ODR`	`0x4800 0014`	Output level, 1 bit for each pin.

Table 1: The three registers that control an output pin of port *A*. Each address is the base address of port *A* plus the offset of the register. `RCC_AHB2ENR` is different: it is in the RCC block at `0x4002 1000`.

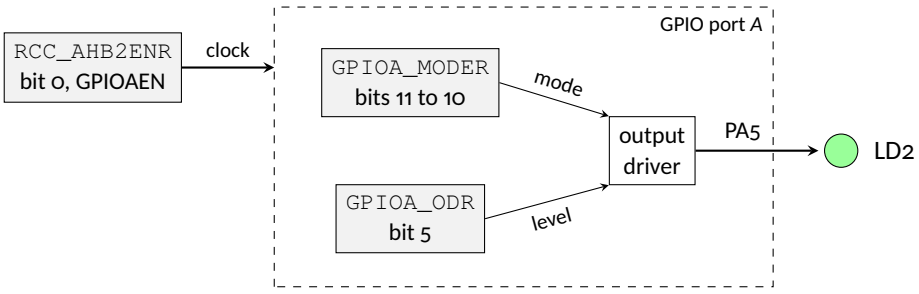


Figure 5: The three registers that make PA5 drive LD2 (simplified). The clock gate switches on the port, `MODER` selects the output mode, and `ODR` gives the level.

Fact 1: The G4 memory map is different from the F4 memory map

On the STM32F4, the GPIO clocks are on `AHB1`, and `GPIOA` starts at `0x4002 0000`. On the STM32G4, the GPIO clocks are on `AHB2`, and `GPIOA` starts at `0x4800 0000`. Most example code on the internet is for the F4. F4 addresses compile on a G4 and cause **no error**. The writes go to the wrong peripheral or to an unused address, and the pin does not change. Compare each address with Table 1.

Fact 2: A peripheral ignores writes until its clock is enabled

A write to `GPIOA_MODER` before GPIOAEN is 1 causes no fault and no warning. The only symptom is a pin that does not change. Generated code can hide this error. The `main()` function that STM32CubeMX generates for this board calls `MX_GPIO_Init()` before the user code. This function enables the GPIOA, GPIOB, GPIOC, and GPIOF clocks. Thus, a program with no clock enable of its own can operate until the generated code changes.

1.5 The mode of a pin

In a register, a *field* is a group of adjacent bits that holds one setting. Each pin n has a 2-bit field in `MODER`, at bits $2n + 1$ to $2n$. PA5 (LD2) uses bits 11 to 10 (Figure 6).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
[31:30]	[29:28]	[27:26]	[25:24]	[23:22]	[21:20]	[19:18]	[17:16]	[15:14]	[13:12]	[11:10]	[9:8]	[7:6]	[5:4]	[3:2]	[1:0]

Figure 6: The fields of `GPIOA_MODER`. The top row gives the pin number, and the bottom row gives the bit range. The field of pin 5 (bits 11 to 10) is orange.

Each field has four values (Figure 7):

- **01, general-purpose output**: the output driver drives the level in `ODR`. The default output type is push-pull. For PA5:

```
"bic   r5, r5, #0xC00     \n"   /* clear bits 11:10 */
"orr   r5, r5, #0x400     \n"   /* set bits 11:10 to 01 */
```

- **00, input**: the level of the pin goes through a Schmitt trigger into the input data register, `GPIOx_IDR`. The value `00` has only zeros, thus it is enough to clear the field:

```
"bic   r5, r5, #0xC00     \n"   /* clear bits 11:10 to 00: input */
```

- **10, alternate function**: an on-chip peripheral drives the pin. `GPIOx_AFRL` and `GPIOx_AFRH` select the peripheral. The generated code uses this mode on PA2 for the virtual COM port, the serial link to the computer through the ST-LINK:

```
"bic   r5, r5, #0x30      \n"   /* clear bits 5:4 (PA2's field) */
"orr   r5, r5, #0x20      \n"   /* set bits 5:4 to 10 */
```

- **11, analog**: the output driver and the Schmitt trigger of the input are off. Thus, the pin uses no power at any voltage on it. This mode is the reset value of most pins, for example PA5. Fact 3 gives the exceptions on port *A*.

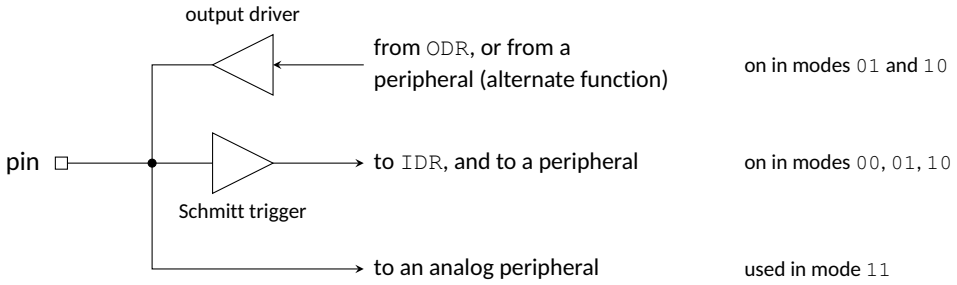


Figure 7: The paths of a GPIO pin (simplified). The field of the pin in `MODER` selects the paths that are on. In analog mode, the output driver and the Schmitt trigger are off.

Analog mode is the reset value to save power. A digital input that nothing drives can float near the threshold of its input circuit. That circuit can then switch again and again, and each switch uses current. In analog mode, an unused pin uses no power.

1.6 Read-modify-write

The field of a pin shares its 32-bit register with the fields of fifteen other pins. An `str` instruction writes all 32 bits. Thus, to change one field, a program does the three steps in Core registers and memory:

1. Load the full register into a core register with `ldr`.
2. Clear the field with `bic`. Then set it with `orr`. The other bits do not change.
3. Store the result with `str`.

This sequence is called *read-modify-write*. The clear is necessary, because `orr` can only set bits. Thus, `orr` cannot change `11` or `10` into `01`. PA5 resets to `11` (analog). Without the `bic`, PA5 stays in analog mode.

After a reset, `GPIOA_MODER` is `0xABFF FFFF`. Table 2 shows the three steps for PA5, with bits 11 to 0 in binary.

Step	Instruction	Bits 11 to 0 of `r5`	`r5`
Read	`ldr r5, [r0]`	`1111 1111 1111`	`0xABFF FFFF`
Modify: clear	`bic r5, r5, #0xC00`	`0011 1111 1111`	`0xABFF F3FF`
Modify: set	`orr r5, r5, #0x400`	`0111 1111 1111`	`0xABFF F7FF`
Write	`str r5, [r0]`		

Table 2: Read-modify-write of the PA5 field, from the reset value. Only bits 11 to 10 change.

Fact 3: One register holds the settings of sixteen pins

`GPIOA_MODER` also holds the fields of these pins:

- PA13, PA14, and PA15, the debug pins. PA13 and PA14 carry Serial Wire Debug (SWD). SWD is the two-wire link that the ST-LINK uses to program and debug the board. The three pins are in alternate function mode after a reset.
- PA2 and PA3, which carry the virtual COM port.

A plain `str` of `0x400` to `GPIOA_MODER` configures PA5 correctly. It also sets all the other fields to `00`. The debug pins leave alternate function mode, and **the debugger disconnects**. Always read, modify, and write back.

The same rule applies to `ODR`.

1.7 Read-modify-write and interrupts

Read-modify-write is not one operation: it has three steps. Another part of the program can change the same register between the load and the store, for example an interrupt handler. The store then writes back the old copy, and that change is lost (Figure 8). Thus, many microcontrollers also have registers that set or clear one pin with a single write. On the STM32, this register is `BSRR`.

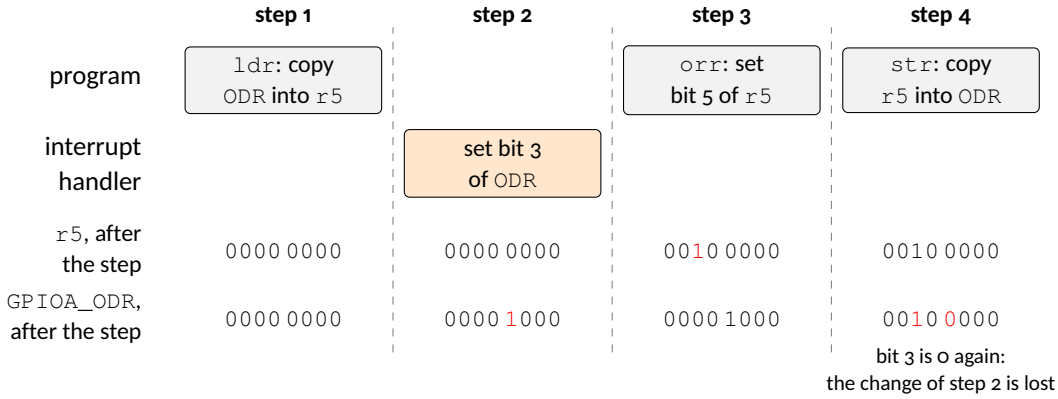


Figure 8: Read-modify-write and an interrupt. The values are bits 7 to 0. A red bit is a bit that the step changes. In step 2, the handler sets bit 3 of `ODR`, but `r5` still holds the copy of step 1. In step 4, the `str` writes back this old copy with bit 5 set, and bit 3 is lost.

1.8 Inline assembly

A C function can contain assembly instructions, in the `__asm volatile` statement of the GNU C compiler. The compiler gives the instructions to the assembler without changes. Thus, the processor executes each instruction of the block. The compiler changes each C statement into instructions such as these. Engineers seldom write a full program in assembly. They often read assembly code: to find a bug, to calculate a time, or to examine what the processor does.

A build of a C project has four steps (Figure 9):

- The preprocessor replaces each `#include` with the file, and each macro with its text.

- The compiler translates C into assembly.
- The assembler translates assembly into machine code.
- The linker puts the code and the variables at their addresses in flash and SRAM, and makes one program file.

For example, the compiler translates the C statement `x |= 0x20;`, for a global variable `x`, into an `ldr`, an `orr`, and an `str`.

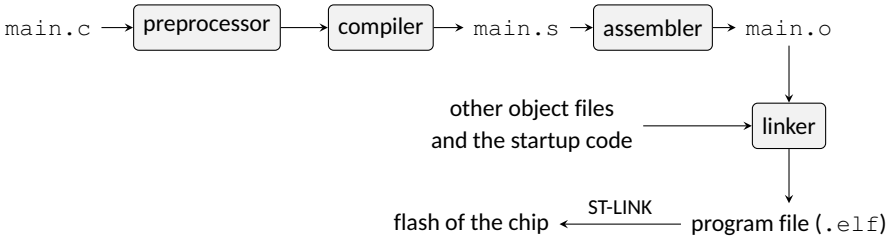


Figure 9: The steps of a build (simplified). The `__asm` blocks go from the compiler to the assembler with no change.

- Each instruction is a C string that ends in `\n`. The compiler joins adjacent strings into one block, with one instruction on each line.
- The keyword `volatile` informs the compiler that it must not move or remove the block. Without `volatile`, the compiler can remove the block, because no C code uses a result of it.
- Three colons separate three lists: the output operands, the input operands, and the *clobber list*. The clobber list names each core register that the block changes, and also `"cc"` (the condition flags) and `"memory"`. The compiler then keeps no value of its own in these registers. A block that uses no C variable leaves the first two lists empty. If the clobber list is not complete, the compiler can keep a value in a register that the block changes. The program can then give wrong results.
- The labels of a block are numbers. `1:` defines a label. `1b` refers to the nearest label 1 before it, and `1f` refers to the nearest label 1 after it. For example, `bne 2b` jumps back to the nearest label 2 before it, if the result is not zero. A numeric label can occur again in the same block. A named label cannot, because the compiler can copy the block to more than one place.
- `ldr rd, =value` loads a 32-bit constant or an address. A 32-bit value does not fit in one instruction. Thus, the assembler stores the value in flash, in a *literal pool*, and the instruction loads it from there. The `.ltorg` line puts the literal pool at the end of the block.

Instruction	Effect
`ldr rd, =value`	Load a 32-bit constant or an address into `rd`.
`ldr rd, [rn, #off]`	Load the word at address `rn + off` into `rd`.
`str rd, [rn, #off]`	Store `rd` at address `rn + off`.
`orr`, `bic`, `eor`	Bitwise OR, bit clear (AND NOT), and exclusive OR.
`subs rd, rn, #n`	Subtract `n` from `rn`, and set the condition flags.
`b`, `bne`	Branch always, and branch if the result is not zero.

Table 3: Some instructions of the Cortex-M4.

1.9 Time in a software delay

The clock of the processor sets the rate at which it executes instructions. At 170 MHz, one clock cycle lasts 1/170 000 000 s, about 5.9 ns. The processor designer publishes the number of cycles of each instruction. Thus, a loop that only counts down can measure time: its duration follows from the number of passes and the cycles of one pass.

The processor reads the next instructions before it executes them. This is the *pipeline*. A branch that jumps is a *taken* branch. After a taken branch, the pipeline holds the instructions after the branch, and not the instructions at its target. The processor discards them and reads the instructions at the target. Thus, a taken branch can take more cycles than a branch that is not taken.

A software delay has three disadvantages:

- Its duration changes when the clock frequency changes.
- Its duration changes when the compiler or the instructions of the loop change.
- The processor can do no other work during the delay.

Thus, most programs measure time with a *hardware timer*: a counter that the clock increments independently of the program. Microcontrollers usually have several timers. The HAL uses one of them, SysTick, to count milliseconds (Time measurement with the HAL).

1.10 Periodic signals and the oscilloscope

A signal that repeats is *periodic*. Its *period* T is the time of one cycle, and its frequency is $1/T$: a period of 20 ms gives 50 Hz. A digital signal that is high for one half of each period and low for the other half has two equal *half-periods* (Figure 10). The *duty cycle* is the part of the period in which the signal is high, here 50 %.

An oscilloscope draws the voltage of a signal as a function of time. Thus, it shows the two levels of a digital signal and the time of each change. The same instrument shows slow signals, for example an LED that blinks, and fast signals, for example the bounce of a contact.

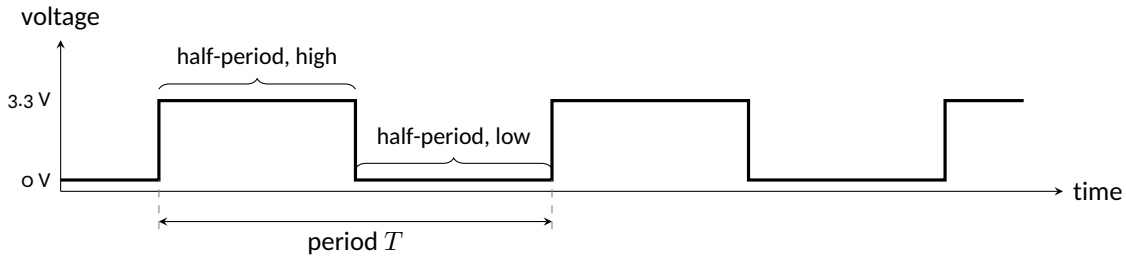


Figure 10: A periodic digital signal with a duty cycle of 50 %. Each period has two equal half-periods.

1.11 Flash, SRAM, and the start of a program

Flash and SRAM are two types of memory with different properties:

- **Flash** keeps its contents without power. It holds the program and the initial values of the global variables. The STM32G491RE has 512 KB of flash. A plain store cannot change it: a write to flash needs an erase sequence and a program sequence.
- **SRAM** loses its contents when the power goes off. A store changes it immediately. It holds the global variables and the *stack*, the memory for the local variables and the return addresses of functions. The STM32G491RE has 112 KB of SRAM.

Thus, the SRAM must be prepared before `main()` starts. After a reset, the processor starts at `Reset_Handler`. STM32CubeMX puts this code in each project for this chip, in the file `startup_stm32g491xx.s`. This code:

- sets the stack pointer to the end of the SRAM, `0x2001 C000`.
- calls `SystemInit()`.
- copies the initial values of the global variables from flash to SRAM.
- writes zeros into the global variables that have no initial value.
- calls `main()`.

Thus, a global variable with no initial value, for example `uint32_t count;`, starts at 0, at an address in SRAM. The linker puts each part of a program in its own *section*, and each section in flash or in SRAM (Figure 11).

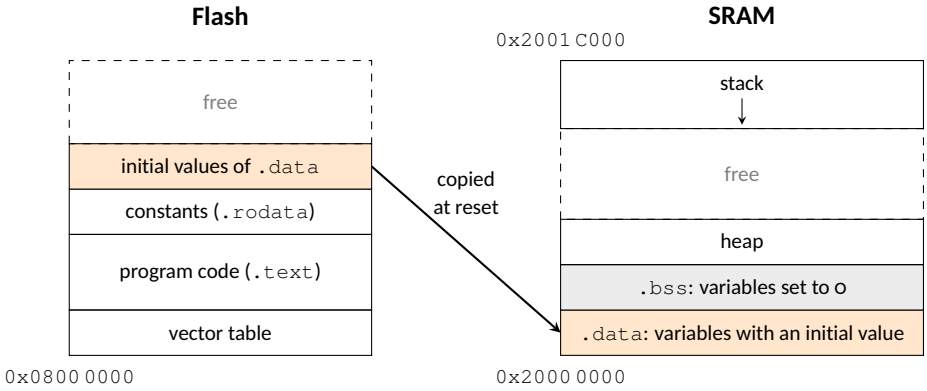


Figure 11: Where a program is in memory (not to scale). At a reset, `Reset_Handler` copies the initial values into `.data` and writes zeros into `.bss`. The stack starts at the end of the SRAM and grows down.

1.12 Buses and clocks

The core does not connect directly to each memory and each peripheral. A *bus matrix* connects the core and the two direct memory access (DMA) controllers to the flash, the SRAM, and the peripherals (Figure 12). A DMA controller copies data between memory and peripherals without the core. The bus matrix lets the core and a DMA controller access different parts of the chip at the same time.

The GPIO ports are on AHB2, a fast bus. The slower peripherals, for example the serial interfaces, are on the two Advanced Peripheral Buses (APB), APB1 and APB2. A bridge connects each APB to the AHB. The clock of an APB can be slower than the clock of the core. With the clock settings that STM32CubeMX generates for this board, all the buses operate at the clock of the core.

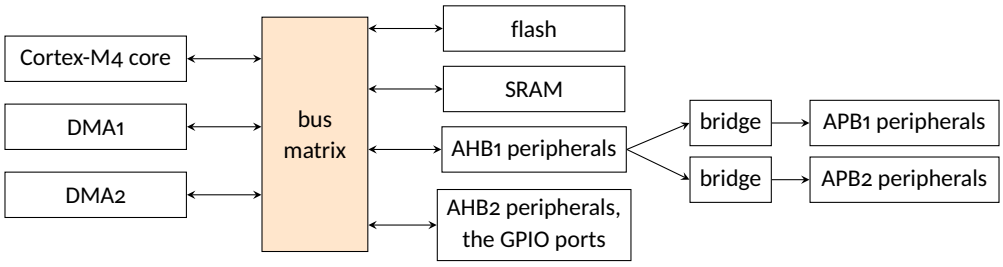


Figure 12: The buses of the STM32G491RE (simplified: the chip has more memories and buses). The core and the two DMA controllers reach the memories and the peripherals through the bus matrix.

Each bus and each peripheral operates with a clock. The clock of the chip comes from an oscillator inside the chip, the HSI, at 16 MHz. A phase-locked loop (PLL) multiplies this frequency. The generated function `SystemClock_Config()` sets $16\ \mathrm{MHz}/4 \times 85/2 = 170\ \mathrm{MHz}$ (Figure 13).

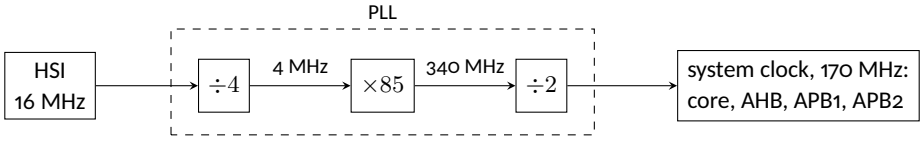


Figure 13: The clock settings that STM32CubeMX generates for this board (simplified).

A clock signal that changes uses power in each part that it reaches, also in a peripheral that does no work. This is the reason for the clock gate of Table 1: the program switches on only the clocks of the peripherals that it uses. This method is called *clock gating*. Many microcontrollers use it.

The flash cannot supply a new instruction in each cycle at 170 MHz. With the generated settings, each flash access takes four extra cycles, the *wait states*. An instruction cache keeps the instructions that the core used recently. Thus, a short loop executes from the cache, with no wait states.

2 Part "B": Register Access in the C Language

To get the state of a button, a program reads from a memory address. The user button B1 is connected to pin PC13. Bit 13 of `GPIOC_IDR`, the input data register of port *C*, shows the level of this pin. The address of `GPIOC_IDR` is `0x4800 0810`. The bit is `1` when button B1 is pressed, and `0` when it is released.

2.1 Register access in the C language

In the C language, a macro changes the address of a register into an expression. The program reads and writes this expression as a variable:

```
#define GPIOC_IDR  (*(volatile uint32_t *)0x48000810UL)
```

- On the right side of an assignment, `GPIOC_IDR` reads the register. On the left side, it writes the register. Each access compiles to an `ldr` or an `str` (Table 3). Thus, `GPIOA_MODER |= ...` is still a read, a modification, and a write.
- `volatile` informs the compiler that each access in the source must occur, in the same sequence. Without `volatile`, the compiler can read an input only one time and use the value again. It can also remove a write if the program does not read the result.

A macro of this type names one register. Production code uses the *device header* of the chip maker: for this chip, `stm32g491xx.h`, which STM32CubeMX puts in each project. It defines a structure for each type of peripheral (Figure 14). For example, `GPIOA->MODER` is the register `GPIOA_MODER`, and `RCC->AHB2ENR` is `RCC_AHB2ENR`. These names follow CMSIS, the standard of Arm for Cortex-M software. Thus, code for Cortex-M chips of other makers has the same form. The HAL uses this header.

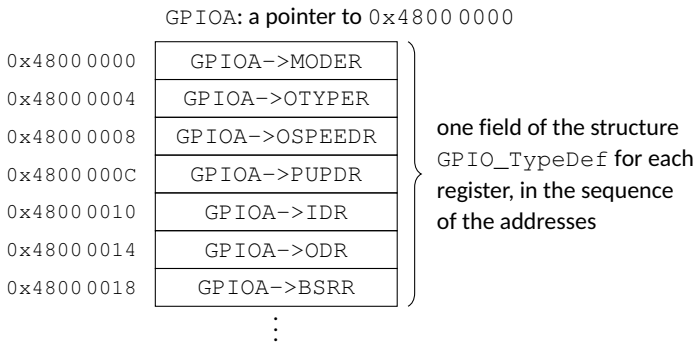


Figure 14: The structure that the device header defines for the registers of port *A*. `GPIOA->ODR` is the word at `0x4800 0014`.

The registers of Table 4 give the input levels, connect a pull-down resistor, and set or clear an output with one write:

- `GPIOx_BSRR` sets or resets individual pins with one write.
- `GPIOx_IDR` holds the level of each input pin.
- `GPIOx_PUPDR` connects an internal pull-up or pull-down resistor to each pin. It has two bits for each pin.

Register	Address	Purpose
`GPIOA_PUPDR`	`0x4800 000C`	Pull-up or pull-down of each port *A* pin.
`GPIOA_IDR`	`0x4800 0010`	Input level of each port *A* pin.
`GPIOA_BSRR`	`0x4800 0018`	Sets or resets individual port *A* pins with one write.
`GPIOC_MODER`	`0x4800 0800`	Mode of each port *C* pin.
`GPIOC_IDR`	`0x4800 0810`	Input level of each port *C* pin.

Table 4: The registers for inputs and for output with one write. Port *C* starts at `0x4800 0800`. `RCC_AHB2ENR` enables its clock, as for port *A*.

2.2 Push-pull outputs

Two switches drive an output pin. One switch connects the pin to ground. The other switch connects the pin to V_{DD}, the supply voltage of the chip (3.3 V on this board). Each switch is a transistor. At all times, one switch is closed and the other switch is open. An `ODR` bit of `0` closes the switch to ground, and the pin is at 0 V. The value `1` closes the switch to V_{DD}, and the pin is at V_{DD} (Figure 15).

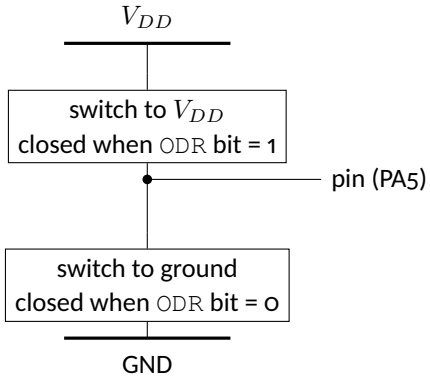


Figure 15: Push-pull output.

Because the pin is always driven, it does not need an external pull resistor. The other output type, open-drain, has only the switch to ground. It needs an external pull-up resistor for a high level. `GPIOx_OTYPER` selects the output type. Its reset value selects push-pull for all pins, thus a push-pull output needs no write to this register.

Open-drain outputs are useful when several devices share one line (Figure 16). Each device can pull the line to ground, and the pull-up resistor holds it high when no device pulls. Buses such as I2C use this method.

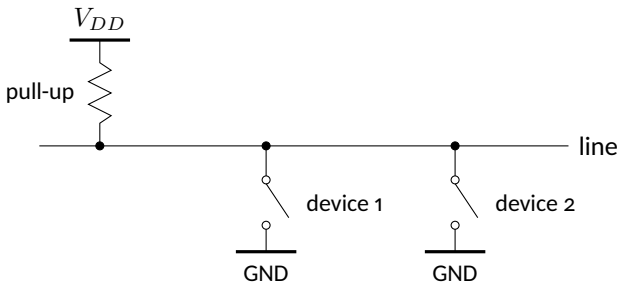


Figure 16: Two open-drain outputs on one line. Each device can pull the line to ground. When no device pulls, the pull-up resistor holds the line high.

`GPIOx_OSPEEDR` sets how fast an output pin changes its level. A slow edge creates less electrical noise. A fast edge is necessary for a signal of high frequency. The reset value of PA5 selects the low speed, which is enough for an LED.

2.3 The Schmitt trigger input

The input circuit compares the voltage of the pin with a threshold voltage. A voltage above the threshold reads `1`, and a voltage below it reads `0`. With one threshold, a slow or noisy signal near the threshold gives a burst of `0`s and `1`s. To prevent this, each STM32G4 GPIO input goes through a *Schmitt trigger*, which has two threshold voltages (Figure 17):

- After a `1`, the input changes to `0` only when the voltage decreases below the lower threshold, V_{IL}.
- After a `0`, the input changes to `1` only when the voltage increases above the higher threshold, V_{IH}.

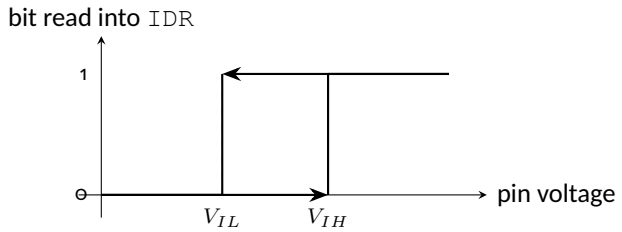


Figure 17: Schmitt trigger hysteresis. A signal between the two thresholds cannot make the bit in `IDR` oscillate.

Hysteresis is common in systems that must select one of two states. A thermostat is an example: the heater goes on below one temperature, and off above a higher temperature. Thus, the heater does not go on and off at each small change of the temperature.

2.4 The input from button B1

The user button B1 is connected to PC13, as Figure 18 shows. Button B1 is **active high**: a press gives a high voltage, which the pin reads as `1`. When button B1 is released, no part drives PC13. An external *pull-down* resistor, which connects the pin to ground, then holds PC13 at 0 V, and the pin reads `0`. When button B1 is pressed, it connects PC13 to V_{DD}, and the pin reads `1`. The level goes through the Schmitt trigger into bit 13 of `GPIOC_IDR`.

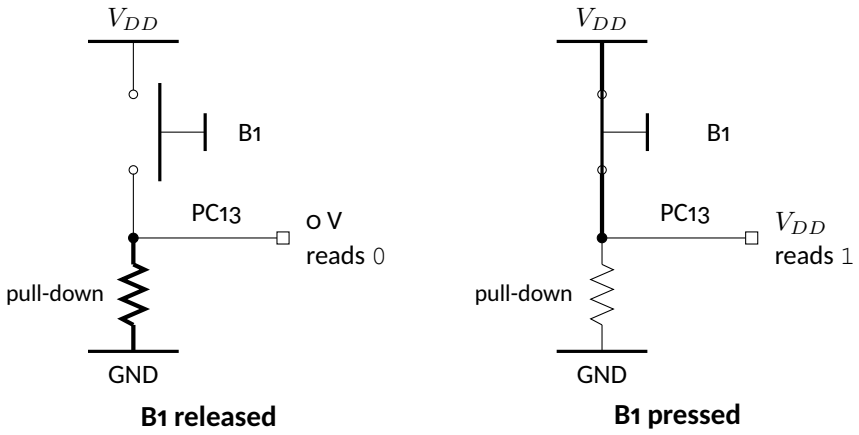


Figure 18: How button B1 sets the level of PC13 (simplified). The thick line shows the path that sets the level of the pin.

Other boards often connect a button the other way: between the pin and ground, with a pull-up resistor. Then a press reads `0`, and the button is *active low* (Figure 19). An input with no pull resistor floats, and it can read `0` or `1` at random. Thus, always find the pull resistor of an input in the schematic of the board, or enable an internal one.

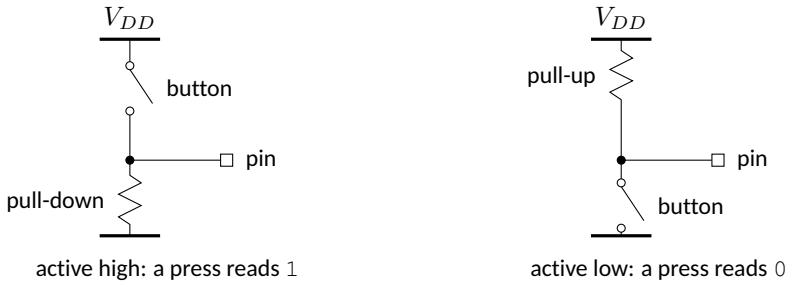


Figure 19: Two ways to connect a button. Button B1 of this board is active high, as on the left.

Each GPIO pin of the STM32 has an internal pull-up resistor and an internal pull-down resistor (Figure 20). `GPIOx_PUPDR` connects one of them, or neither, to the pin. Thus, an input can have a defined level with no resistor on the board.

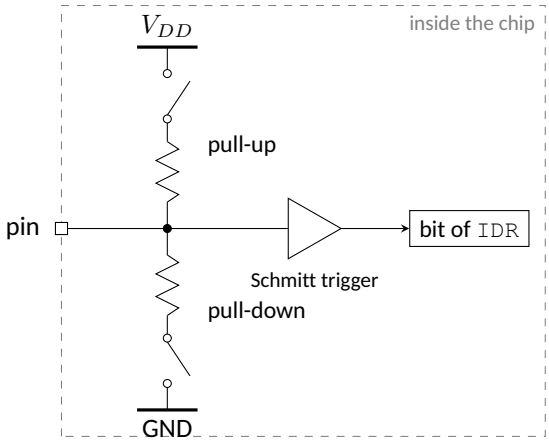


Figure 20: The internal pull resistors of a GPIO pin (simplified). `GPIOx_PUPDR` closes one of the two switches, or neither.

There are two different quantities (Figure 21):

- the *level*: the state of button B1 at this time, as bit 13 shows it.
- a *press*, or *rising edge*: the time at which the level changes from `0` to `1`.

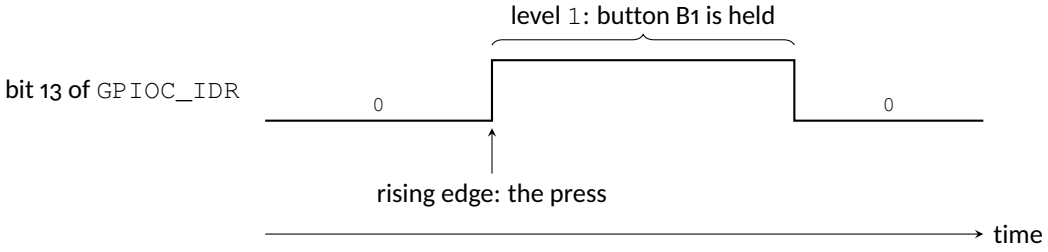


Figure 21: The level and the rising edge of one press of button B1.

A program that uses the level does its action again and again while button B1 is held. A program that uses the edge does its action one time for each press. To find an edge, a program compares each value with the previous value, which it keeps in a variable. This method assumes that one press gives exactly one rising edge. A mechanical button does not always give one edge.

2.5 Contact bounce

When a mechanical switch closes, the elasticity of the contact material makes the contacts separate and connect again many times, for a short period. Then a stable connection occurs. Thus, the voltage at the pin changes between ground and V_{DD} several times before it becomes stable at its final level (Figure 22). A similar transient occurs when the switch opens. This is *contact bounce*, and it occurs in all mechanical switches. Its duration is different for different switches, and for different operations of the same switch.

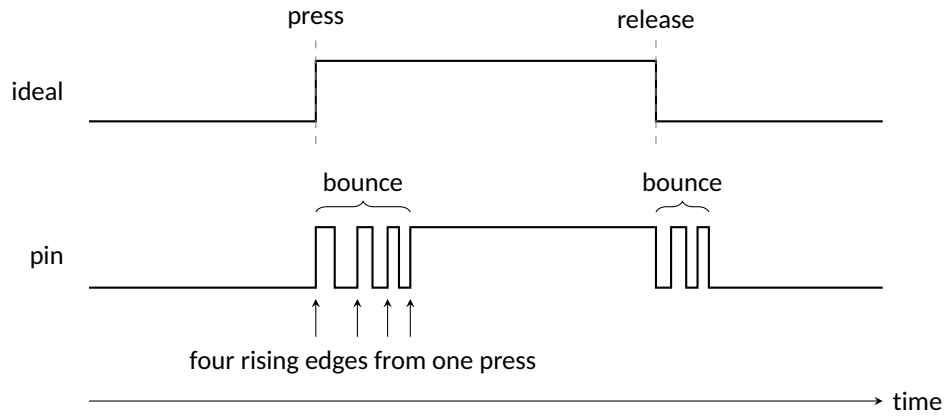


Figure 22: Contact bounce (not to scale). The ideal signal changes level one time at each press and one time at each release. The real signal at the pin changes between the two levels several times before it is stable.

The Schmitt trigger corrects a slow or noisy signal near a threshold. It does not remove contact bounce: each bounce is a full transition between ground and V_{DD}, and it crosses the two thresholds. Thus, the input data register records each bounce as a valid change of level. A program that reads the pin often enough detects each bounce. As a result, an edge detector counts one press as several presses.

The bounce of a switch reaches the pin only if nothing filters it. Button B1 of this board has a filter: a resistor and a capacitor between the button and PC13 (Methods to remove contact bounce). Thus, the bounce of button B1 seldom reaches PC13. On the pin of a switch with no filter, an oscilloscope shows the bounce.

2.6 Methods to remove contact bounce

There are two methods to remove contact bounce:

- **Hardware filter.** A resistor (resistance R) and a capacitor (capacitance C) between the contact and the pin make a low-pass filter. The voltage of the pin then changes slowly: the *time constant* $\tau = RC$ gives the time of a change. A bounce that is much shorter than τ ends before the voltage crosses the input threshold. Thus, it does not reach the input data register.
- **Software debounce.** The program accepts a new input level only after the level stays the same for longer than the bounce.

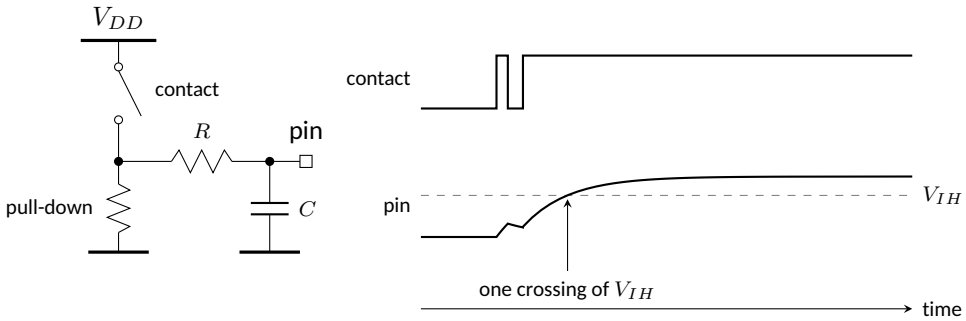


Figure 23: A hardware filter (not to scale). The contact bounces, but the voltage of the pin rises slowly and crosses the threshold one time.

Button B1 of this board has this circuit. The resistor R27 (330 Ω) is R, the capacitor C35 (100 nF) is C, and R26 (100 kΩ) is the pull-down. When the contact closes, C35 charges through R27, with τ of approximately 33 μs. When the contact opens for a short time during the bounce, C35 can discharge only through R26 and R27, with τ of approximately 10 ms. Thus, a short break of the contact seldom pulls PC13 below the threshold. These values of τ are calculated from the values of the parts.

Each method has a cost. A hardware filter needs parts on the board, and its time constant delays each change. A software debounce costs code and time of the processor, and it also delays each accepted change. Other software methods exist. A program can read the input at a fixed interval, for example every few milliseconds, and accept a level after several equal values. A program can also ignore the input for a short time after each accepted edge.

Figure 24 shows a software debounce with a counter. On each pass of the loop, the program compares the input with the accepted level. Each pass on which they are different adds 1 to a counter. Each pass on which they are the same sets the counter to 0. When the counter reaches a limit N, the program accepts the new level. Thus, a bounce sets the counter to 0 again, and only a stable level makes the counter reach N.

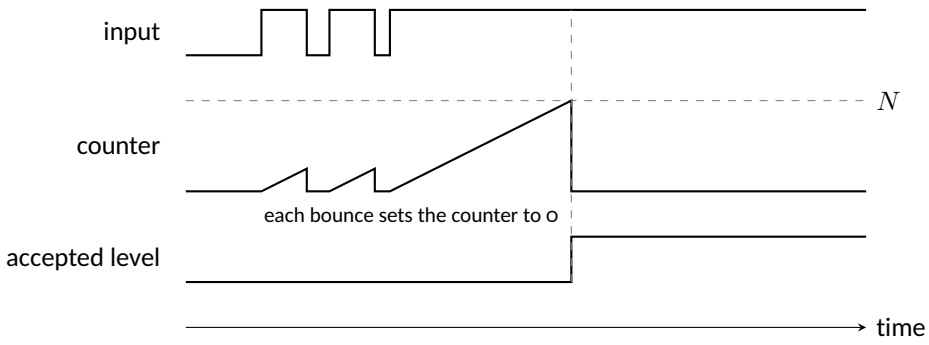


Figure 24: A software debounce with a counter (not to scale). The accepted level changes only when the input stays stable for N passes.

A program that reads an input on each pass of its loop uses *polling*. It detects a change only when it reads the input. Thus, a long pass delays the response. The other method is an *interrupt*. The hardware requests an interrupt when an event occurs, for example a change of an input. The processor then stops the program between two instructions and executes a function, the *interrupt handler*. After the handler, the program continues from the point where it stopped (Figure 25).

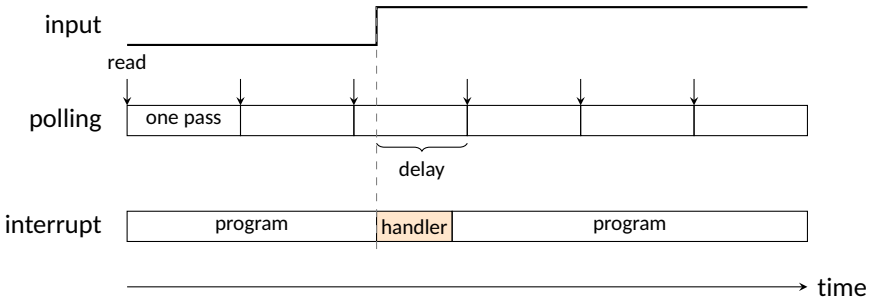


Figure 25: Polling and an interrupt. With polling, the program detects the change at its next read. With an interrupt, the handler starts a short time after the change, at any point of the program.

3 Part "C": The HAL

A library can do the register accesses for a program. The functions of the HAL configure, read, and write the registers of the GPIO ports, and they also measure time. For example, a program can measure the duration of each press of a button.

3.1 The HAL

The hardware abstraction layer (HAL) is the library of C functions from ST for the peripherals of the STM32. Table 5 gives some of its functions, for GPIO pins and for time. The HAL has two advantages:

- **Abstraction.** The program gives a port and a pin. The function finds the registers and the bit positions, and accesses them.
- **Portability.** ST supplies a HAL for each STM32 family. Each HAL has the same function names, with the register addresses of its family. Thus, a program that uses the HAL for the G4 moves to a different STM32 with few changes. For register-level code, the programmer must examine each address again, as Fact 1 shows for the F4.

Function	What it does	Registers
`__HAL_RCC_GPIOx_CLK_ENABLE()`	Enables the clock of port x	`RCC_AHB2ENR`
`HAL_GPIO_Init()`	Configures pins: mode, pull, output type, speed	`MODER`, `PUPDR`, ...
`HAL_GPIO_ReadPin()`	Returns the level of a pin	`IDR`
`HAL_GPIO_WritePin()`	Sets or clears a pin	`BSRR`, `BRR`
`HAL_GetTick()`	Returns the time since reset, in milliseconds	none

Table 5: Some HAL functions for GPIO pins and for time. `HAL_GPIO_WritePin()` sets a pin through `BSRR` and clears it through `BRR`, the bit reset register of the port.

The HAL is one layer in a stack of software layers (Figure 26):

- the registers, which Parts "A" and "B" access directly.
- the device header, with the CMSIS names of the registers.
- the low-layer (LL) drivers of ST: short inline functions that access one register or one field.
- the HAL: functions that configure a peripheral or do a full operation.
- frameworks that operate on chips of many makers, for example Arduino, with functions such as `digitalWrite()`.

Each layer hides more of the hardware than the layer below it. A higher layer needs less code in the program. A lower layer gives a smaller and faster program, and it shows what the hardware does. For example, `HAL_GPIO_Init()` examines each bit of its pin mask in a loop. For an output pin, it does a read-modify-write on four registers: `OSPEEDR`, `OTYPER`, `PUPDR`, and `MODER`.

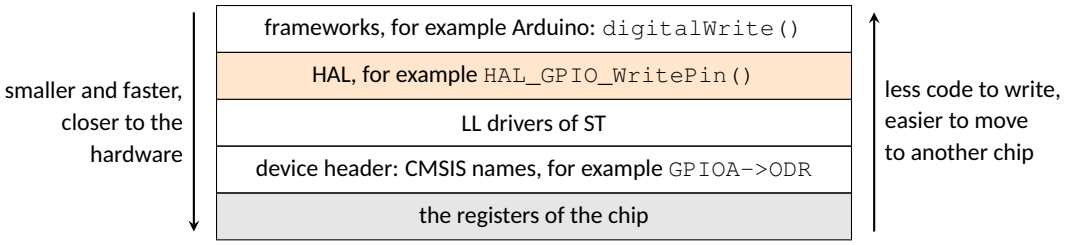


Figure 26: Layers of software for a microcontroller (simplified). A higher layer hides more of the hardware.

3.2 Time measurement with the HAL

The time base of the HAL is the SysTick timer of the Cortex-M4. `HAL_Init()` starts this timer. The timer then increments a counter each millisecond, independently of the program. `HAL_GetTick()` returns the value of the counter, a `uint32_t`. To measure an interval, record the counter when the event starts. At a later time, subtract the recorded value from the counter:

```
uint32_t start = HAL_GetTick();              /* the event starts */
uint32_t elapsed = HAL_GetTick() - start;    /* later: time in ms */
```

The two variables must be `uint32_t`, as the counter. The subtraction is then unsigned, and it gives the correct interval also after the counter goes back to 0.

SysTick is a 24-bit timer inside the Cortex-M4 processor. Arm defines it, thus each Cortex-M4 chip has the same SysTick, at the same address. It counts down at the clock frequency of the core. At 0, it loads its reload value again and requests an interrupt. The HAL sets the reload value for one interrupt each millisecond, thus each 170 000 cycles at 170 MHz. The interrupt handler, `SysTick_Handler()` in `stm32g4xx_it.c`, calls `HAL_IncTick()`, which adds 1 to the counter. Thus, the counter is a variable in SRAM, and the timer sets its rate (Figure 27).

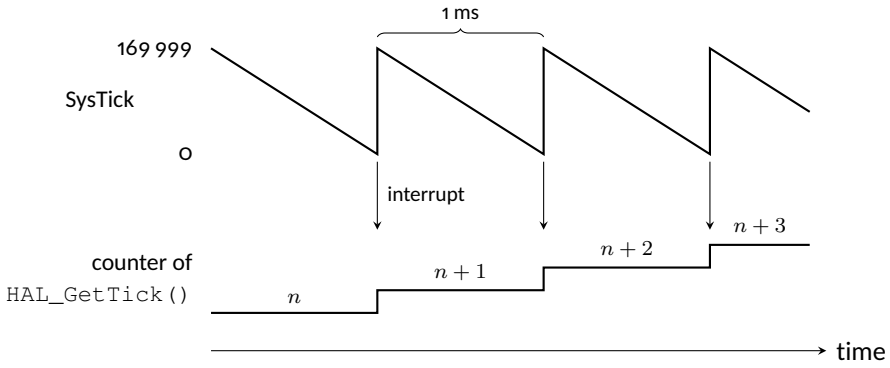


Figure 27: SysTick at 170 MHz. It counts down from 169 999 to 0 in 1 ms, and then it requests an interrupt. Each interrupt adds 1 to the counter of `HAL_GetTick()`.

A `uint32_t` counts modulo 2^{32}: after the largest value, it goes back to 0. Each counter of fixed width does the same. For example, the cycle counter of the Cortex-M4, when it is on, adds 1 at each clock cycle, and it is also 32 bits wide. At 170 MHz, it goes back to 0 after approximately 25 s.

3.3 A loop that does not wait

`HAL_Delay()` waits: it returns only after the time is over. During the wait, the program cannot read an input or change an output (Figure 28). A program that must read an input during the interval does not wait. On each pass of its loop, it subtracts the time of an event from `HAL_GetTick()`, and it compares the result with a limit. It acts only when the result reaches the limit. Thus, each pass is short, and the loop can do other work on each pass. For example, this loop toggles LD2 each 500 ms, and it can also read button B1 on each pass:

```
uint32_t last = HAL_GetTick();         /* time of the last toggle */
while (1)
{
    if (HAL_GetTick() - last >= 500U) /* 500 ms after the last toggle */
    {
        last += 500U;                   /* the time of this toggle */
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
    /* other work, for example: read button B1 */
}
```

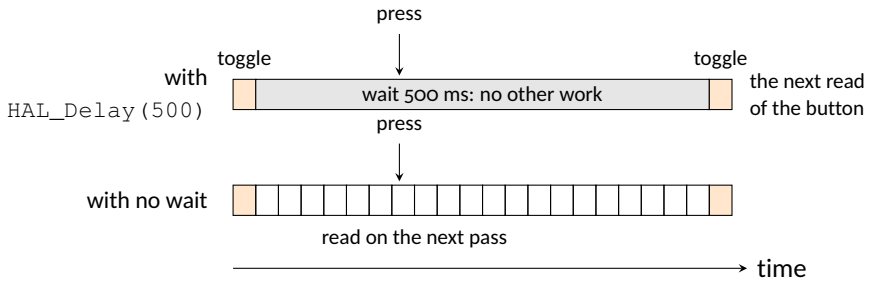


Figure 28: A loop that waits and a loop that does not wait. During the wait, the program does not read the button. The loop with no wait reads it on each short pass.

Appendix A: Morse code

Morse code shows each letter as a sequence of two signals: a short *dot* and a long *dash* (Table 6). All durations are in units of one dot:

- The dash is three units long.
- The pause between the signals of one letter is one unit long.
- The pause between letters is three units long.

Figure 29 shows the letters A and T as presses of a key.

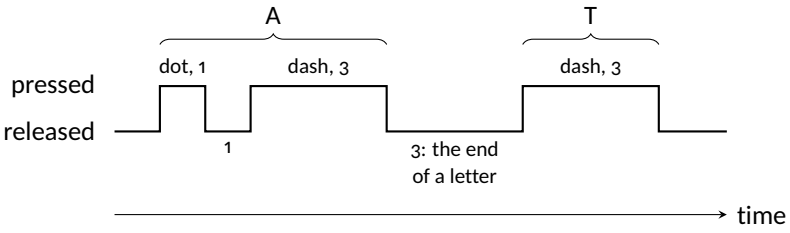


Figure 29: The letters A and T in Morse code. The numbers are durations in units of one dot.

A	.−	**H**	....	**O**	−−−	**V**	...−
B	−...	**I**	..	**P**	.−−.	**W**	.−−
C	−.−.	**J**	.−−−	**Q**	−−.−	**X**	−..−
D	−..	**K**	−.−	**R**	.−.	**Y**	−.−−
E	.	**L**	.−..	**S**	...	**Z**	−−..
F	..−.	**M**	−−	**T**	−		
G	−−.	**N**	−.	**U**	..−		

Table 6: The International Morse code for the letters A to Z. The dot is . and the dash is −.

Morse code is a variable-length code. The letters that occur most often in English have the shortest codes: E is one dot, and T is one dash. Modern data compression, for example the Huffman code, uses the same idea.